

Linux y el interprete comandos

Dr. José Luis Quiroz Fabián

1 Introducción a Linux y sus distribuciones

Linux es un sistema operativo de código abierto basado en Unix, creado originalmente por Linus Torvalds en 1991. A diferencia de otros sistemas operativos comerciales, Linux se distribuye bajo la licencia GNU GPL, lo que permite a cualquier persona utilizar, modificar y distribuir su código fuente de forma libre.

Linux es ampliamente utilizado en servidores, supercomputadoras, sistemas embebidos, dispositivos móviles (como Android) y también en equipos de escritorio.

Dado que el núcleo de Linux (llamado kernel) solo proporciona las funcionalidades básicas, distintas comunidades y organizaciones han creado distribuciones, o distros, que combinan el kernel con herramientas, entornos gráficos, gestores de paquetes y otras utilidades.

Cada distribución tiene un propósito o enfoque distinto: algunas son ideales para usuarios novatos, otras para servidores, educación, seguridad informática o desarrollo de software. Ejemplos populares incluyen:

- Debian: Conocida por su estabilidad y filosofía de software libre.
- Ubuntu: Muy usada en entornos educativos y personales. Amigable para principiantes. Basada en Debian.
- Fedora: Similar a Ubuntu pero basada en RedHat.
- Arch Linux: Para usuarios avanzados que desean un control total del sistema.
- Kali Linux: Enfocada en pruebas de penetración y análisis forense digital.

2 Principales directorios del sistema Linux

El sistema de archivos en Linux sigue una jerarquía estándar definida por el Filesystem Hierarchy Standard (FHS). A continuación se presentan los directorios más importantes:

/ (raíz): Directorio principal del sistema. Todos los demás directorios cuelgan de este.

/bin Contiene los ejecutables esenciales del sistema accesibles por todos los usuarios. Incluye comandos básicos como `ls`, `cp`, `mv`, etc.

/boot Almacena los archivos necesarios para el arranque del sistema, como el kernel y el gestor de arranque.

/dev Contiene archivos de dispositivos que representan hardware del sistema.

/etc Archivos de configuración del sistema. Es crucial para el desarrollo web cuando se configuran servidores como Nginx, Apache o bases de datos.

/home Directorios personales de los usuarios. Aquí se desarrollan y almacenan proyectos en muchos casos.

/lib Bibliotecas compartidas necesarias para ejecutar programas en **/bin** y **/sbin**.

/media y

/mnt Puntos de montaje temporales para dispositivos externos como discos duros y memorias USB.

/opt Software adicional instalado manualmente. Algunos frameworks o entornos de desarrollo pueden instalarse aquí.

/proc Sistema de archivos virtual que expone información del kernel y procesos en tiempo real.

/root Directorio personal del usuario **root**.

/run Archivos de estado del sistema desde el arranque.

/sbin Comandos del sistema usados principalmente por el administrador (**root**).

/srv Contiene datos para servicios del sistema como **ftp**, **http**, etc. Muy relevante si se alojan sitios web directamente.

/tmp Archivos temporales. Pueden ser usados por aplicaciones web para almacenar archivos subidos o sesiones.

/usr Aplicaciones de usuario y sus dependencias. Contiene subdirectorios como **/usr/bin**, **/usr/lib**, **/usr/share**.

/var Archivos variables como logs, colas de impresión, cachés y archivos temporales de aplicaciones. Es vital para monitoreo y diagnóstico de aplicaciones web.

3 Interprete de comando

El intérprete de comandos permite la comunicación entre el sistema y el usuario. Aunque actualmente las interfaces gráficas de usuario (GUI) facilitan el trabajo cotidiano, todavía existen funciones que se resuelven mejor desde la línea de comandos. Algunas ventajas de los interpretes de comando son las siguientes:

- Menor consumo de recursos (muy importante si se actúa sobre sistemas remotos mediante una conexión lenta).
- Posibilidad de programar scripts para automatizar tareas administrativas.

Existe una gran variedad de interpretes de comandos, algunos ejemplos son los siguientes:

- Bourne Again Shell (bash)
- Korn Shell (ksh)

- C Shell (csh)
- Bourne Shell (sh)
- command.com
- cmd.exe

En la siguiente sección se muestra algunos comandos que podemos ejecutar en el interprete *bash*.

4 Comandos

Las Tablas 1 y 2 muestran algunos ejemplos de comandos en Bash. Para cada comando se muestra, el nombre del comando, su descripción, algunos de sus parámetros y un ejemplo.

Comando	Descripción	Parámetros	Ejemplo
ls	Lista los archivos y directorios.	• -a: Todos los archivos • -l: Listar con detalles • -R: Listar recursivamente los sub-directorios	ls -l
cd	Permite cambiar de directorio.	• ~ : Regresa al Home • ..: Regresa a un directorio atrás.	cd ..
pwd	Permite conocer la ubicación actual en el sistema de archivos.		
clear	Limpia pantalla.	• -x: No elimina el contenido del scrollbar	clear -x
cat	Muestra el contenido de un archivo de texto.	• -n: Muestra el número de línea. • -e: Muestra un \$ al final de cada línea.	cat -n file.txt
more	Muestra el contenido de un archivo de texto por partes.	• -n: Número de líneas a desplegar	more -n 2 file.txt

Tabla 1: Ejemplos de comandos (Parte 1)

Comando	Descripción	Parámetros	Ejemplo
cp	Copiar un archivo o directorio.	• -r: Copia de forma recursiva (para directorios) • -i: Copia de forma interactiva	cp -r directorio rutaACopiar
ps	Despliega información de los procesos actuales	• -e: Muestra todos los procesos. • -f: Muestra información detallada de los procesos. • -o: Muestra los procesos en un formato dado por el usuario.	ps -eo user,pid
top	Despliega una fotografía de los procesos en tiempo real.	• -U: Muestra los procesos de un usuario	ps -U invitado
man	Despliega la ayuda de un comando.		man ls

Tabla 2: Ejemplos de comandos (Parte 2)

5 Parámetros de un comando

Los comandos como varios programas pueden recibir parámetros que permiten cambiar su comportamiento por default o bien su salida en pantalla. Algunos parámetros de comando son:

5.1 ls

¿Qué se muestra al ejecutar los siguientes comandos?:

1. ls -l
2. ls -a
3. ls -la

5.2 date

¿Qué se muestra al ejecutar los siguientes comandos?:
macOS

1. date -v +1d

2. `date -v -1w`
3. `date +%A- %B- %Y`

Ubuntu

1. `date -d "tomorrow"`
2. `date -d "6 weeks ago"`
3. `date +%A- %B- %Y`

5.3 touch

¿Qué se muestra al ejecutar los siguientes comandos?:

Ubuntu

1. `touch -m -date="anio-mes-dia" archivo`

Ubuntu

5.4 apt-get

Comando para instalar paquetes en sistemas basados en Debian (ejecutar como root o con sudo). ¿Qué se muestra al ejecutar los siguientes comandos?:

```
sudo apt-get install cmatrix
```

5.5 Tuberías y redireccionamiento

Las tuberías permiten que la salida de un comando sea la entrada de otro. El símbolo para representar las tuberías en el bash es `|`. Ejemplo:

```
1 ls -l | wc -l
```

Muestra la cantidad de líneas que despliega el comando `ls`.

Ubuntu:

```
1 grep "MemTotal" /proc/meminfo | awk '{print $2}'
```

Filtra la salida de:

```
1 grep "MemTotal" /proc/meminfo
```

y muestra únicamente la segunda columna.

macOS:

```
1 sysctl -n hw.memsize | awk '{print $0/1024/1024/1024 " GB"}'
```

Filtra la salida de:

```
1 sysctl -n hw.memsize
```

Tomando el valor obtenido de sysctl, lo divide por 1024 tres veces (para convertir de bytes a gigabytes) y luego agrega "GB." al final para indicar la unidad.

En una terminal compile y ejecute (gcc arbol.c -o arbol & ./arbol) el siguiente programa:

```
1 #include <sys/types.h>
2 #include <unistd.h>
3 #include <stdio.h>
4
5 int main(){
6     int i;
7     printf("PID: %d\n",getpid());
8     for(i=0;i<4;i++){
9         if(fork()){
10             if(fork()){
11                 break;
12             }
13         }
14     }
15     sleep(1000);
16     return 0;
17 }
```

Después, en otra terminal ejecute la secuencia de comandos:

```
1 ps -eo pid,user,comm | grep "arbol" | head -n 1 | awk {'print $1'} | xargs ./
  pstree.sh
```

¿Qué observa?

6 Salida a un archivo

Para enviar la salida de un comando a un archivo se utiliza el símbolo >. Ejemplo:

```
1 date > salida
```

Si la salida se desea enviar a un archivo existente se utiliza:

```
1 date >> salida
```

7 Script

Un *script* es un archivo de texto que contiene comandos a ejecutar. Se ejecuta de forma similar a un ejecutable C, usando ./ . La primer línea del script hace referencia al interprete de comando.

```
1 #!/bin/bash
```

Si la ruta es incorrecta, se puede utilizar el comando *whereis* para buscar el ejecutable:

```
1 whereis bash
```

Posteriormente se tienen que asignarle permisos de ejecución, lo cual se puede realizar de la siguiente forma:

```
1 chmod +x nombre del archivo script
```

7.1 Ejemplo 1: Hola mundo

Escribir un archivo llamado hola.sh y agregarle el texto:

```
1 #!/bin/bash
2 echo "Hola mundo"
```

¿Qué se muestra al ejecutar el script?

7.2 Ejemplo 2: basico.sh

Ejemplo básico con comandos.

```
1 #!/bin/bash
2
3
4 if [ "$#" -ne 1 ]; then
5     echo "Uso: $0 <nombre_dir>" >&2
6     exit 1
7 fi
8
9 DIR="$1"
10 FILE="info.txt"
11
12 # 1) crear directorio si no existe
13 mkdir -p "$DIR" || { echo "No se pudo crear $DIR" >&2; exit 2; }
14 echo "Directorio creado/confirmado: $DIR"
15
16 # 2) entrar al directorio
17 cd "$DIR" || { echo "No se puede acceder a $DIR" >&2; exit 3; }
18
19 # 3) mostrar directorio actual
20 echo "Ahora en: $(pwd)"
21
22 # 4) crear un archivo simple con cat
23 cat > "$FILE" <<EOF
24 Este es el archivo $FILE
25 Creado por: $(whoami)
26 Ruta actual: $(pwd)
27 Fecha: $(date '+%F %T')
28 EOF
29
30 echo "Archivo creado: $FILE"
31
32 # 5) mostrar contenido con cat
33 echo
34 echo "Contenido de $FILE:"
35 cat "$FILE"
```

```
36
37 exit 0
```

7.3 Ejemplo 3: descarga.sh

Escribir un archivo llamado `descarga.sh` y agregarle el texto:

```
1 #!/bin/bash
2
3 # Valores por defecto
4 URL_DEFAULT="cdn.mindmajix.com/blog/images/what-is-linux-120619.png"
5 LOGFILE_DEFAULT="./download.log"
6
7 VERBOSE=0
8 DIR=""
9 LOGFILE=""
10
11 usage() {
12     #Similiar a echo, pero para varias lineas
13     cat <<EOF
14     -v, --verbose    Modo verbose (imprime mensajes en pantalla además de log)
15     -h, --help      Mostrar esta ayuda
16     Si no se proporciona logfile, se usa: $LOGFILE_DEFAULT
17     EOF
18 }
19
20 # Parseo manual simple de opciones
21 for arg in "$@"; do
22     if [ "$arg" = "-v" ] || [ "$arg" = "--verbose" ]; then
23         VERBOSE=1
24     elif [ "$arg" = "-h" ] || [ "$arg" = "--help" ]; then
25         usage
26         exit 0
27     else
28         # argumentos posicionales: primero directorio, luego logfile
29         if [ -z "$DIR" ]; then
30             DIR="$arg"
31         elif [ -z "$LOGFILE" ]; then
32             LOGFILE="$arg"
33         else
34             # ignorar argumentos extra
35             :
36         fi
37     fi
38 done
39
40 # Validar directorio (obligatorio)
41 if [ -z "$DIR" ]; then
42     echo "Error: Debe indicar el nombre del directorio a utilizar." >&2
43     usage
44     exit 1
45 fi
46
47 # Rellenar valores por defecto
48 if [ -z "$URL_DEFAULT" ]; then
49     echo "Error: Debe indicar la url a utilizar." >&2
50     exit 1
51 fi
52
53 if [ -z "$LOGFILE" ]; then
```

```

54 LOGFILE="$LOGFILE_DEFAULT"
55 fi
56
57 if [ -z "$LOGFILE" ]; then
58     echo "Error: Debe indicar el archivo log." >&2
59     exit 1
60 fi
61
62 # Función de log simple
63 log() {
64     local msg="$1"
65     local line
66     line="$(date '+%F %T') - $msg"
67     if [ "$VERBOSE" -eq 1 ]; then
68         echo "$line" | tee -a "$LOGFILE"
69     else
70         echo "$line" >> "$LOGFILE"
71     fi
72 }
73
74 log "Inicio: directorio='$DIR' logfile='$LOGFILE'"
75
76 # Crear el directorio si no existe
77 if [ -e "$DIR" ]; then
78     log "Ok: existe el directorio: $DIR"
79 else
80     mkdir -p "$DIR" 2>/dev/null
81     if [ $? -eq 0 ]; then
82         log "Creando el directorio: $DIR"
83     else
84         log "ERROR: No se pudo crear el directorio: $DIR"
85         exit 2
86     fi
87 fi
88
89 # Acceder al directorio
90 cd "$DIR" 2>/dev/null
91 if [ $? -ne 0 ]; then
92     log "ERROR: No se pudo acceder al directorio: $DIR"
93     exit 1
94 fi
95 log "Accediendo al directorio: $DIR"
96
97 # Nombre del archivo a guardar
98 FILE_NAME="$(basename "$URL_DEFAULT")"
99
100 # Descargar con wget (silencioso si no verbose)
101 if [ "$VERBOSE" -eq 1 ]; then
102     log "Descargando (verbose): $URL_DEFAULT -> $FILE_NAME"
103     wget "$URL" -O "$FILE_NAME"
104 else
105     log "Descargando: $URL_DEFAULT -> $FILE_NAME"
106     wget -q "$URL_DEFAULT" -O "$FILE_NAME"
107 fi
108
109 WGET_EXIT=$?
110 if [ $WGET_EXIT -ne 0 ]; then
111     log "ERROR: Archivo no descargado (wget exit=$WGET_EXIT): $URL_DEFAULT"
112     # volver al directorio original (por claridad)
113     cd - >/dev/null 2>&1
114     exit $WGET_EXIT
115 else

```

```

116 log "Archivo descargado correctamente: $(pwd)/$FILE_NAME"
117 fi
118
119 # Volver al directorio original (opcional)
120 cd - >/dev/null 2>&1
121
122 log "Fin del script."
123 exit 0

```

¿Qué se ocurre al ejecutar el script?

7.4 Ejemplo 4: ciclos.sh

Escribir un archivo llamado ciclo.sh y agregarle el texto:

```

1 #!/bin/bash
2 # Demostración de estructuras de repetición en Bash
3 set -euo pipefail
4
5 # --- Configuración inicial ---
6 MAX="${1:-6}" # Máximo por defecto (puede pasarse como argumento)
7
8
9
10 # 1) while: contador ascendente
11 echo "while (contador ascendente)"
12 n=1
13 while [ "$n" -le "$MAX" ]; do
14     printf "while: %d\n" "$n"
15     ((n++))
16 done
17
18 # 2) until: contador descendente (ejecuta hasta que la condición sea verdadera)
19 echo "until (contador descendente)"
20 m=$MAX
21 until [ "$m" -le 0 ]; do
22     printf "until: %d\n" "$m"
23     ((m--))
24 done
25
26 # 3) for clásico (estilo C): índices y uso de continue/break
27 echo "for (( ... )) con continue y break"
28 for ((i=1; i<=MAX; i++)); do
29     if (( i % 2 == 0 )); then
30         echo "for: $i (par) continue"
31         continue # salta la impresión siguiente para números pares
32     fi
33     if (( i == 5 )); then
34         echo "for: $i alcanzado 5, break"
35         break # corta el ciclo cuando i == 5
36     fi
37     echo "for: $i (impar)"
38 done
39
40
41 # 4) ciclo anidado: tabla de multiplicar pequeña
42 echo "Ciclo anidado: tabla de multiplicar 1..5"
43 for ((r=1; r<=5; r++)); do
44     for ((c=1; c<=5; c++)); do

```

```

45     printf "%2d" $((r*c))
46     done
47     printf "\n"
48 done

```

¿Qué se muestra al ejecutar el script?

7.5 Ejemplo 5: conexion.sh

Considere que trabaja con el servidor `pacifico.izt.uam.mx` y tiene el usuario `so24p`. Escribir un archivo llamado `conexion.sh` y agregarle el texto:

```

1  #!/bin/bash
2
3  # Configuración básica
4  REMOTE_USER="so2025II"
5  REMOTE_HOST="pacifico.izt.uam.mx"
6  SSH_KEY="./so_llave" # Clave privada específica del usuario
7  REMOTE_COMMAND="ls -l ~so2025II" # Comando a ejecutar en el servidor
8  OUTPUT_FILE="resultado.txt" # Archivo local donde se guardará el
9  resultado
10
11 # Ejecutar comando remoto y guardar resultado usando la clave SSH específica
12 ssh -i "${SSH_KEY}" -o IdentitiesOnly=yes "${REMOTE_USER}@${REMOTE_HOST}" "${
13     REMOTE_COMMAND}" > "${OUTPUT_FILE}"
14
15 # Verificar si el comando remoto fue exitoso
16 if [ $? -eq 0 ]; then
17     echo "Comando ejecutado exitosamente. Resultado guardado en ${OUTPUT_FILE}"
18 else
19     echo "Error al ejecutar el comando en el servidor remoto."
20 fi

```

¿Qué se muestra al ejecutar el script?

8 Ejercicios

Resolver los siguientes problemas mediante script de `bash`. La salida de cada script debe ser FORMATEADA para que sea amigable, es decir, no solo se trata de desplegar la información.

1. Verificación de servicios remotos

Implementar un script de monitoreo remoto que, sin requerir privilegios de `root`, verifique en `pacifico.izt.uam.mx` si un conjunto de servicios (por ejemplo, `nginx` y `apache2`) se encuentra activo, utilizando preferentemente `systemctl is-active` y, en su defecto, `service <svc>status` o detección de procesos con `ps`; los resultados deberán registrarse en un archivo JSON nombrado `log_servicios_YYYY-MM-DD.json`, el cual debe incluir un encabezado con la fecha y, por cada servicio verificado, un estado normalizado (`activo`, `inactivo`, `no_conectado` o `error`) acompañado de un breve detalle y el método empleado. Los servicios se deben especificar en un archivo `json`:

```
{
```

```

    "servicios": [
        "nginx",
        "apache2",
        "mysql",
        "postgresql"
    ]
}

```

El archivo resultado debe ser como:

```

{
  "fecha": "2025-09-04",
  "host": "pacifico.izt.uam.mx",
  "resumen": {
    "total": 2,
    "activo": 1,
    "inactivo": 1,
    "no_conectado": 0,
    "error": 0
  },
  "verificaciones": [
    {
      "servicio": "nginx",
      "metodo": "systemctl",
      "estado": "activo",
      "detalle": "active",
      "checked_at": "2025-09-04T18:30:02Z",
      "duracion_ms": 110
    },
    {
      "servicio": "apache2",
      "metodo": "ps",
      "estado": "inactivo",
      "detalle": "sin procesos coincidentes",
      "checked_at": "2025-09-04T18:30:03Z",
      "duracion_ms": 90
    }
  ]
}

```

2. Respaldo automático de proyectos

Implementar un script que genere un respaldo comprimido del directorio `~/proyecto`, almacenándolo en la carpeta `~/respaldos` con el formato de nombre `proyecto_YYYY-MM-DD.tar.gz`, donde la fecha corresponde al día de ejecución. El script deberá además integrar un mecanismo de depuración que elimine automáticamente los respaldos con más de 7 días de antigüedad, de modo que se optimice el uso del espacio en disco y se asegure la disponibilidad de copias recientes del proyecto. Todos los resultados de la ejecución deberán registrarse en una bitácora en formato JSON llamada `bitacora_respaldo_YYYY-MM-DD.json`, ubicada

en `~/respaldos`, indicando el respaldo generado, los respaldos eliminados y el estado final de la operación. Ejemplo de archivo bitácora:

```
{
  "fecha": "2025-09-04",
  "operacion": "respaldo_proyecto",
  "respaldo_generado": {
    "archivo": "proyecto_2025-09-04.tar.gz",
    "ruta": "/home/usuario/respaldos/proyecto_2025-09-04.tar.gz",
    "tamano_bytes": 20485760,
    "estado": "exitoso",
    "duracion_seg": 3.2
  },
  "respaldos_eliminados": [
    "proyecto_2025-08-27.tar.gz",
    "proyecto_2025-08-26.tar.gz"
  ],
  "estado_final": "completado"
}
```

- `respaldo_generado`: detalla el archivo creado, ruta, tamaño, estado y duración.
- `respaldos_eliminados`: lista de respaldos borrados por antigüedad.
- `estado_final`: indica si el proceso se completó correctamente.

3. Monitoreo de integridad con SHA256

Implementar un script que calcule los valores **SHA256** de archivos críticos del sistema definidos en un archivo de configuración en formato **JSON** (por ejemplo, `config_archivos.json`), el cual contendrá la lista de rutas de los archivos a monitorear como `/etc/passwd`, `/etc/shadow` o archivos de configuración relevantes. El script deberá comparar los valores obtenidos con los previamente almacenados para detectar modificaciones; en caso de discrepancias, se deberá generar una alerta y registrar el evento, de modo que se cuente con un mecanismo de monitoreo que contribuya a la detección temprana de alteraciones no autorizadas. Este script deberá ejecutarse automáticamente a intervalos regulares mediante **cron**, de manera que el monitoreo sea continuo sin intervención manual. Los resultados de cada ejecución deberán registrarse en una bitácora en formato **JSON**, cuyo nombre seguirá el esquema `log_hashes_YYYY-MM-DD.json` (por ejemplo, `log_hashes_2025-09-04.json`), indicando los archivos verificados, sus valores actuales, los valores esperados y el estado final de la verificación.

Ejemplo de `config_archivos.json`:

```
{
  "archivos": [
    "/etc/passwd",
    "/etc/shadow",
    "/etc/ssh/sshd_config",
    "/etc/fstab"
  ]
}
```

Ejemplo de bitácora generada en una ejecución `log_hashes_2025-09-04.json`.

```
{
  "fecha": "2025-09-04",
  "operacion": "verificacion_hashes",
  "archivos": [
    {
      "ruta": "/etc/passwd",
      "sha256_actual": "3c8f48f8c5d44e1f5e9f3e8e9a6a40d54b7c0f7d9c0f7e5c3a2e8b0e3a5b7f4c",
      "sha256_esperado": "3c8f48f8c5d44e1f5e9f3e8e9a6a40d54b7c0f7d9c0f7e5c3a2e8b0e3a5b7f4c",
      "estado": "sin_cambios"
    },
    {
      "ruta": "/etc/shadow",
      "sha256_actual": "a7f9b18e5f0f44b1c3f0d8c1a5e9b9e6c7d4e5f0f9a1b2c3d4e5f6a7b8c9d0e1",
      "sha256_esperado": "f1e2d3c4b5a6978877665544332211ffeeddccbbaa99887766554433221100ff",
      "estado": "modificado"
    }
  ],
  "resumen": {
    "total_verificados": 2,
    "sin_cambios": 1,
    "modificados": 1
  },
  "alerta": true,
  "estado_final": "completado_con_alertas"
}
```

- fecha: día de ejecución (nombre del archivo debe coincidir).
- ejecuciones[]: permite guardar varias corridas en un mismo día.
- estado: puede ser `sin_cambios` o `modificado`.
- alerta: se activa si al menos un archivo resultó modificado.